Отношения между классами и объектами

Александра Волосова,
к.т.н., доцент кафедры
ИУ5

Презентация разработана в рамках гранта «Грант на обучение студентов по образовательным программам высшего образования для топ-специалистов в сфере информационных технологий. Договор № 70-2025-000850 с АНО «Аналитический центр при Правительстве Российской Федерации»

Часть 1

CTAD

CTAD (Class Template Argument Deduction)

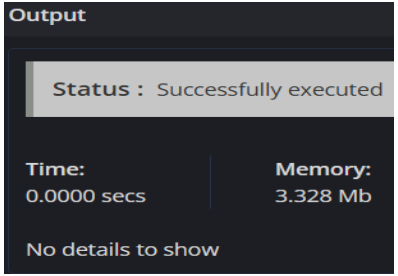
- правило языка C++ (начиная со стандарта **C++17)**, которое позволяет компилятору **автоматически определять типы шаблонных аргументов шаблона** класса (class template) на основе типов аргументов его конструктора

- до C++17, при создании объекта классового шаблона нужно было **явно указывать все шаблонные параметры** в угловых скобках

```
#include <vector>
#include <utility>

int main() {
    // указание на int
    std::vector<int> v1 {1, 2, 3};

    // указание типов для пары
    std::pair<int, double> p1 {42, 3.14};

    // указание типов для мьютекса в
   // lock_guard
    std::mutex m;
    std::lock_guard<std::mutex> lock(m);
    return 0;
}
```

C++17

```
#include <vector>
#include <utility>
#include <mutex>
int main() {
    // CTAD: выводит std::vector<int>
    std::vector v1 {1, 2, 3};
    // CTAD: выводит std::pair<int, double>
    std::pair p1 {42, 3.14};

    // CTAD: выводит std::lock_guard<std::mutex>
    std::mutex m;
    std::lock_guard lock(m); // С C++17 работает
(CTAD)
    return 0;
}
```



Где удобно использовать:

1. Сложные итераторы: При создании диапазонов

```
std::vector data = {1, 2, 3, 4, 5};
// до C++17:
std::reverse_iterator<std::vector<int>::iterator> first(data.rbegin());
// автоматический вывод шаблонных параметров классовых шаблонов из аргументов конструктора.
//1. data.rbegin() имеет тип std::reverse_iterator<std::vector<int>::iterator>
// 2. first должен быть std::reverse_iterator<std::vector<int>::iterator>
std::reverse_iterator first(data.rbegin());
```

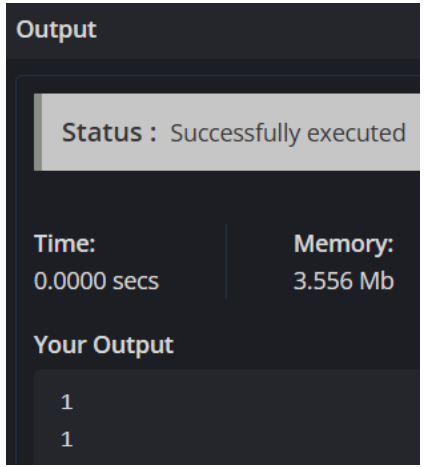
2. Умные указатели:

```
// до C++17:
std::shared_ptr<int> ptr(new int(10));

// C++17. CTAD
std::shared_ptr ptr(new int(10));
}
```

3. **Обобщенный код**: В шаблонных функциях, где тип контейнера зависит от типа переданных аргументов

Сложные итераторы

```
#include <iostream>
#include <vector>
#include <iterator>
int main() {
    std::vector data = {1, 2, 3, 4, 5};
    std::reverse_iterator<std::vector<int>::iterator> old(data.rbegin());
    std::reverse_iterator first(data.rbegin());
    std::cout << (typeid(old) == typeid(first)) << std::endl;  // 1
    std::cout << (*old == *first) << std::endl;                // 1
    return 0;
}
```



std::cout << (typeid(old) == typeid(first)) << std::endl;

old и **first** имеют **одинаковый тип**.

typeid(old) возвращает информацию о типе переменной old

typeid(first) возвращает информацию о типе first

Сравнение **==** возвращает true (1), если типы одинаковы

std::cout << (*old == *first) << std::endl;

Значения, на которые указывают **old** и **first**, **равны**.

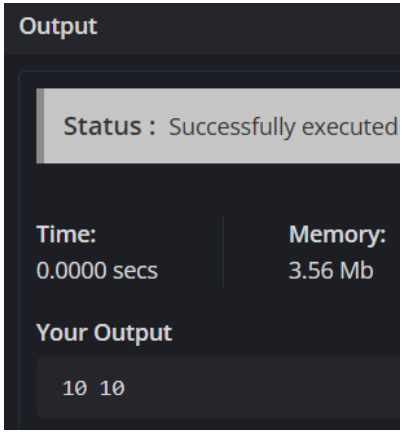
*old разыменовывает итератор old — получает первый элемент (с конца)

*first делает то же самое для first

Оба указывают на элемент 5, поэтому сравнение дает true (выводится как 1)

Умные указатели

```
#include <memory>
#include <iostream>
int main() {
   // до 17++
   std::shared_ptr<int> ptr1(new int(10));
   // C++17. CTAD
std::shared_ptr ptr2 = std::make_shared<int>(10);
std::cout << *ptr1 << " " << *ptr2 << std::endl;
  return 0;
}
```



Обобщенный код

Обобщенный код —позволяет работать с **любыми типами данных**, а не с конкретными (например, int, double, string).

```
#include <vector>
#include <list>
#include <iostream>
#include <typeinfo>
template<typename Container>
void example(const Container& c) {
   // До C++17 - явное указание типа
   std::vector<typename Container::value_type> old(c.begin(), c.end());
   // C++17 - CTAD выводит тип автоматически
   std::vector vec(c.begin(), c.end());
   std::cout << "Типы одинаковы? "
         << (typeid(old) == typeid(vec)) << std::endl;
   // Размеры векторов
   std::cout << "old size: " << old.size() << std::endl;
   std::cout << "vec size: " << vec.size() << std::endl;
   // Первые элементы
   if (!old.empty() && !vec.empty()) {
      std::cout << "old[0]: " << old[0] << std::endl;
      std::cout << "vec[0]: " << vec[0] << std::endl;
   }
   std::cout << "------------------------" << std::endl;
}
```

```
int main() {
   std::list<int> lst = {1, 2, 3};
   std::cout << "=== Тест с int ===" << std::endl;
   example(lst);  // vec = vector<int>
   std::list<double> lst2 = {1.1, 2.2, 3.3};
   std::cout << "=== Тест с double ===" << std::endl;
   example(lst2); // vec = vector<double>
   return 0;
}
```

Output

Status : Successfully executed

Time: 0.0000 secs

Memory: 3.824 Mb

Your Output

```
=== Тест с int ===
Типы одинаковы? 1
old size: 3
vec size: 3
old[0]: 1
vec[0]: 1
------------------------
```

```
=== Тест с double ===
Типы одинаковы? 1
old size: 3
vec size: 3
old[0]: 1.1
vec[0]: 1.1
```

Обобщенный код. Пример

```
// Функция только для int
int max_int(int a, int b) {
   return (a > b) ? a : b;
}

// Функция только для double
double max_double(double a, double b) {
   return (a > b) ? a : b;
}

// Функция только для string
std::string max_string(std::string a, std::string b)
{
   return (a > b) ? a : b;
}
```

```
template<typename T>
T max(T a, T b) {
   return (a > b) ? a : b;
}

// Работает для любого  типа:
int x = max(10, 20);          // T = int
double y = max(3.14, 2.71);     // T = double
std::string s = max("abc", "xyz"); // T = string
```

Библиотека STL

```
std::vector<int> v1;       // Вектор для int
std::vector<double> v2;     // Вектор для double
std::vector<std::string> v3; // Вектор для string
// алгоритм sort работает с любым вектором
std::sort(v1.begin(), v1.end());
std::sort(v2.begin(), v2.end());
```

CTAD в C++17 и в C++20

В C++17 CTAD **не работал для шаблонных классов-наследников** и **некоторых агрегатных типов**:

```
template<typename T>
// Wrapper наследуется от T
struct Wrapper : T {
   // конструктор не объявлен - это агрегат
};
struct Data {
   int a;
};
// В C++17 - ошибка: «не удается вывести аргументы шаблона»
Wrapper w{Data{10}};
template<typename T>
struct Wrapper : T {};

struct Data { int a; };

// Возможное исправление ошибки в C++17
template<typename T>
Wrapper(T) -> Wrapper<T>;  // Указание
компилятору: если передали T, то Wrapper<T>

Wrapper w{Data{10}};  // C++17: T = Data
```

C++20

```
#include <iostream>

template<typename T>
struct Base {
   T value;
};
template<typename T>
struct Derived : Base<T> {};
int main() {
   // Способ 1: Явный Base<int> с designated initializer
   Derived d1{Base<int>{.value = 10}};

   // Способ 2: CTAD для Base, потом для Derived
   Base b{.value = 20};       // C++20: CTAD выводит Base<int>
   Derived d2{b};             // C++17+: CTAD выводит Derived<int>

   // Способ 3: Все в одной строке с auto
   auto d3 = Derived{Base{.value = 30}};
   // Проверяем
   std::cout << d1.value << std::endl;  // 10
   std::cout << d2.value << std::endl;  // 20
   std::cout << d3.value << std::endl;  // 30

   return 0;
}
```

```
10
20
30

...Program finished with exit code 0
Press ENTER to exit console.
```

компилятор автоматически выводит типы Base<int> и Derived<int>

Base ctad_base{20} - CTAD для простых агрегатов

Derived ctad_derived{Base{30}} - CTAD для наследников (C++20)

Агрегат: рабочий вариант в C++17:

```
// для агрегата Base<T> с одним полем T value, инициализатор {10} означает, что T =  int
template<typename T>
struct Base {
   T value;
};
Base b{10};
//для агрегата Base<T> с одним полем T value, инициализатор {10} означает, что T =  int
//Ошибка при использовании наследника. От какого типа T унаследован Derived?

template<typename T>
struct Derived : Base<T> {
   // пусто
};
Derived d{Base<int>{10}};
```

Безошибочная работа CTAD в C++17:

- **Агрегаты без наследования** — компилятор видит поля напрямую
- **Классы с явными конструкторами** — компилятор анализирует параметры конструктора

С чем корректно работает CTAD в C++17:

- Агрегаты без наследования

```
template<typename T>
struct Base {
   T value;
};

Base b{10};  // C++17: компилятор видит поле value типа T -> T = int
```

- Классы с явными конструкторами

```
template<typename T>
struct Holder {
   T data;
   Holder(T d) : data(d) {}  // Явный конструктор
};

Holder h{42};  // C++17: работает через анализ конструктора
```

Часть 2

std::optional. Концепты и альтернативы наследованию

- **CTAD** (Class Template Argument Deduction)

  — автоматический вывод типов

- **Наследовании в шаблонных классах**

  — проблемы C++17 и решения C++20

- **Использование сложных иерархий внутри библиотек**

std::optional

- **CTAD** — автоматический вывод типов
- **CTAD важен для шаблонных классов**
- **Контейнеры** — шаблонные классы
- **std::optional** — контейнер (хранит 0 или 1 элемент)
- **Реализация optional** внутри использует сложные техники
- **Одна из техник** — наследование от шаблонного параметра

std::optional **применяется для:**

- **Поиска в массиве** — когда элемент может не найтись
- **Парсинга чисел** — когда строка может не быть числом
- **Полей в структурах** — которые могут отсутствовать (например, отчество)
- **Функции** — которые могут не вернуть результат

std::optional —**иллюстрация** того, как:

- Шаблонные классы используют наследование внутри

std::optional<T> — это "умный" контейнер, который либо хранит значение типа T, либо ничего не хранит

Проблема, которую решает optional:

```
// Как вернуть "нечто" из функции?
int find_value(bool flag) {
    if (flag) return 42;   // есть значение
    else ???               // 1? 0? NULL?
}

// C++17 решение:
std::optional<int> find_value(bool flag) {
    if (flag) return 42;    // есть значение
    else return std::nullopt; // ничего нет
}
```

Связь с наследованием (C++17)

- В C++17 std::optional внутри мог использовать сложные иерархии наследования, чтобы:
  - Не вызывать конструкторы лишний раз
  - Правильно управлять временем жизни объекта
  - Оптимизировать память

Примерная внутренняя структура:

```
// Упрощенно, как это могло выглядеть в C++17
template<typename T>
class optional {
    struct empty_state {};     // состояние "пусто"
    struct storage : empty_state, T { ... }; // сложное наследование
};
```

Концепты (C++20)

В C++17 std::optional внутри использовал **сложные иерархии наследования** для эффективной работы. В C++20 концепты упростили реализацию

```
// Упрощенная реализация optional в стиле C++17
// иерархия наследования для реализации std:: optional
struct Empty {};
template<typename T>
class Optional : private Empty {
// наследование для оптимизации
union { T value; char dummy; };
   bool has_value;

   // Много вспомогательных классов...
public:
   Optional(T v) : value(v), has_value(true) {}
   T& operator*() { return value; }
};
```

```
// Та же функциональность, но с концептами
template<typename T>
class Optional {
   union { T value; char dummy; };
   bool has_value;

public:
   // Концепт обозначает требования к T
   template<typename U> requires
std::constructible_from<T, U>
   Optional(U&& v) : value(std::forward<U>(v)),
has_value(true) {}

   T& operator*() { return value; }
};
```

Связь с концептами (C++20)

- C++20: **концепты** (concepts) — способ наложить ограничения на шаблонные параметры, не требуя наследования от определённого базового класса

```
template<typename T>
// T должен быть копируемым
concept Copyable = requires(T a) { T(a); };

template<Copyable T>  // T должен удовлетворять концепту Copyable
class optional {
   // Реализация без сложных иерархий
   // Можно использовать более прямые методы
};
```

Традиционный ООП-подход:

```
class MyClass : public ISerializable { ... };
```

Современный подход (C++20):

```
class MyClass { public: void serialize(); };
template<Serializable T> void process(T& obj) {
obj.serialize();
}
```

MyClass не обязан наследовать интерфейс, достаточно соответствия концепту.

ООП-подход – наследование

```
// Базовый класс для всех, кто умеет говорить
struct Speaker {
    virtual void speak() const = 0;
};

// Наследники
class Dog : public Speaker {
public:
    void speak() const override { std::cout << "Гав!\n"; }
};

class Cat : public Speaker {
public:
    void speak() const override { std::cout << "Мяу!\n"; }
};

// Функция требует наследника Speaker
template<typename T>
void make_speak(const T& animal) {
    static_assert(std::is_base_of_v<Speaker, T>);
    animal.speak();
}
```

Концепты обеспечивают **структурную типизацию** (structural typing) в шаблонах: требования предъявляются к **интерфейсу типа**, а не к его положению в иерархии наследования.

Важна **способность типа выполнять определенные операции**, а не его происхождение от конкретного базового класса.

C++20 подход - концепты

```
#include <concepts>
#include <iostream>
// Концепт: "умеет говорить" — есть метод speak()
template<typename T>
concept CanSpeak = requires(T obj) {
    obj.speak();
};
// Любой тип с методом speak() подходит!
template<CanSpeak T>
void make_speak(const T& animal) {
    animal.speak();
}
// Обычные классы без наследования
class Dog {
public:
    void speak() const { std::cout << "Гав!\n"; }
};
class Cat {
public:
    void speak() const { std::cout << "Мяу!\n"; }
};
int main() {
    Dog dog;
    Cat cat;

    make_speak(dog);  // Гав!
    make_speak(cat);  // Мяу!
}
```

Часть 3

C++20/23

consteval и constinit (C++20)

consteval — функция, гарантированно выполняющаяся на этапе компиляции.
constinit — гарантирует инициализацию статических переменных на этапе компиляции.

```cpp
#include <iostream>
#include <array>
// consteval: гарантированное вычисление на этапе компиляции
consteval int square(int x) {
    return x * x;
}
// Иерархия классов с consteval функцией
class MathConstant {
public:
    consteval MathConstant(int v) : value(v) {}
    consteval int getValue() const { return value; }
private:
    int value;
};
class Circle : public MathConstant {
public:
    // consteval конструктор
    consteval Circle(int radius) : MathConstant(radius) {}
    consteval double area() const {
        return 3.14159 * getValue() * getValue();
    }
};
// ========== constinit (статическая инициализация) ==========
constinit std::array<int, 5> global_array = {1, 2, 3, 4, 5};
constinit int global_counter = 0;
class Config {
public:
    static constinit inline int version = 1;
    static constinit inline std::array<int, 3> defaults = {10, 20, 30};
};
```

```cpp
int main() {
    std::cout << "=== consteval ===" << std::endl;

    // Все вычисления на этапе компиляции
    constexpr Circle c1(10);
    constexpr double area1 = c1.area();

    std::cout << "Circle area with radius 10: " << area1 << std::endl;

    // Можно использовать в константных выражениях
    std::array<int, square(5)> arr;  // массив из 25 элементов
    std::cout << "Array size from square(5): " << arr.size() << std::endl;

    // consteval нельзя вызвать во время выполнения
    // int x = 5;
    // int y = square(x);  // Ошибка!

    std::cout << "\n=== constinit ===" << std::endl;

    // Гарантированная инициализация до main()
    std::cout << "Global array[0]: " << global_array[0] << std::endl;
    std::cout << "Config version: " << Config::version << std::endl;
    std::cout << "Config defaults[1]: " << Config::defaults[1] << std::endl;

    // constinit можно изменять
    global_counter++;
    Config::version = 2;

    std::cout << "Updated global_counter: " << global_counter << std::endl;
    std::cout << "Updated config version: " << Config::version << std::endl;

    return 0;
}
```

Your Output

```
=== consteval ===
Circle area with radius 10: 314.159
Array size from square(5): 25

=== constinit ===
Global array[0]: 1
Config version: 1
Config defaults[1]: 20
Updated global_counter: 1
Updated config version: 2
```

std::variant и std::visit (C++17)

Альтернатива полиморфным иерархиям для случаев, когда набор типов фиксирован и известен заранее

Вместо:

```
Shape* s = getShape();
s->draw(); // Виртуальный вызов
```

Можно использовать:

```
std::variant<Circle, Square> v = getShape();
std::visit([](auto& shape) {
shape.draw();
}, v);
```

<u>Преимущества variant:</u>

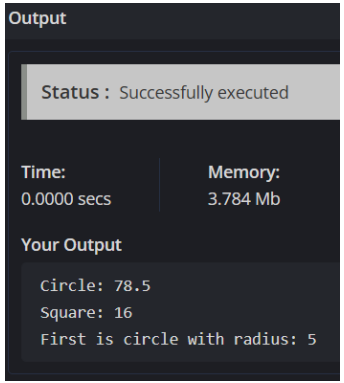
· Быстрее (нет виртуальных вызовов)
· Хранится по значению (нет проблем с временем жизни)
· Безопаснее (тип всегда известен)

<u>Недостатки</u>:

· Фиксированный набор типов (нельзя добавить новый тип без изменения кода)

```cpp
#include <variant>
#include <vector>
#include <iostream>

struct Circle {
   double radius;
   double area() const { return 3.14 * radius * radius; }
   std::string name() const { return "Circle"; }
};

struct Square {
   double side;
   double area() const { return side * side; }
   std::string name() const { return "Square"; }
};

// variant с фиксированным набором типов
using Shape = std::variant<Circle, Square>;

// Вспомогательные visitor'ы
struct AreaVisitor {
   double operator()(const auto& shape) const { return shape.area(); }
};

struct NameVisitor {
   std::string operator()(const auto& shape) const { return shape.name(); }
};
```

```cpp
int main() {
   std::vector<Shape> shapes = {Circle{5.0}, Square{4.0}};

   for (const auto& shape : shapes) {
      std::cout << std::visit(NameVisitor(), shape) << ": "
               << std::visit(AreaVisitor(), shape) << std::endl;
   }

   // Доступ к конкретному типу без dynamic_cast:
   if (auto* circle = std::get_if<Circle>(&shapes[0])) {
      std::cout << "First is circle with radius: " << circle->radius;
   }
}
```



Type Erasure (Стирание типа):

Type Erasure — техника, позволяющая работать с объектами разных несвязанных типов через единый интерфейс, скрывая детали реализации.

Классические примеры:

· std::function — может хранить любой вызываемый объект
· std::any — может хранить значение любого типа

· Небольшой уровень внутреннего наследования
· Виртуальные вызовы скрыты от пользователя
· Предоставляется семантика значения (value semantics)

Преимущества:

· Гибкие API
· Отсутствие жесткой привязки к иерархии классов
· Сохранение семантики значения

Run | Debug | Stop | Share | Save | { } Beautify | Language C++ 23

main.cpp

```cpp
        void draw() const override { obj.draw(); }
    };

    std::unique_ptr<Concept> pimpl;

public:
    template<typename T>
    Drawable(T obj) : pimpl(std::make_unique<Model<T>>(obj)) {}

    void draw() const { pimpl->draw(); }
};

// Разные несвязанные типы с методом draw()
struct Circle { void draw() const { std::cout << "Circle\n"; } };
struct Square { void draw() const { std::cout << "Square\n"; } };
struct Text   { void draw() const { std::cout << "Text\n"; } };

int main() {
    std::vector<Drawable> drawings;
    drawings.push_back(Circle{});
    drawings.push_back(Square{});
    drawings.push_back(Text{});

    for (const auto& d : drawings) {
        d.draw();  // Единый интерфейс
    }
}
```

input

```
Circle
Square
Text


...Program finished with exit code 0
Press ENTER to exit console.
```

std::move_only_function (C++23)

Улучшенная версия std::function, которая поддерживает move-only типы (например, std::unique_ptr). Это еще один шаг в развитии type erasure, делающий работу с уникальными ресурсами в полиморфных обертках более естественной.

Run Debug Stop Share Save { } Beautify Language C++ 23

main.cpp

```cpp
#include <functional>  // для move_only_function
#include <memory>
#include <iostream>

class UniqueResource {
    std::unique_ptr<int> data;
public:
    UniqueResource(int v) : data(std::make_unique<int>(v)) {}
    void use() const { std::cout << *data << std::endl; }
};

int main() {
    std::move_only_function<void()> f2 = [res = UniqueResource(42)] {
        res.use();
    };

    f2();  // ДОЛЖНО вывести 42
    return 0;
}
```

input

```
42


...Program finished with exit code 0
Press ENTER to exit console.
```

C++ → C++23:

1. Упрощение синтаксиса (CTAD, using enum)
2. Повышение безопасности (концепты, constinit)
3. Улучшение производительности (consteval, variant)
4. Гибкость без потери эффективности (концепты вместо наследования)
5. Переход от жестких иерархий классов к более гибким, безопасным и эффективным решениям на основе шаблонов и концептов